



TECHNICAL PROJECT DOCUMENTATION

TripEase

AI-Powered Smart Tourism & Personalized Travel Intelligence Platform

"Your Journey. One Platform."

Team NaviGo

Project Title	TripEase
Domain	AI-based Tourism Planning and Analytics
Guide	Mr. Anjith Raja
Department	Computer Science and Engineering

MEMBERS

Name	USN
Dhanushree G	4AL23CS041
Chaithanya K	4AL23CS032

September 2026

TABLE OF CONTENTS

1. Introduction.....	4
1.1 Background.....	4
1.2 Motivation.....	4
1.3 Scope of the Document.....	4
1.4 Organisation of the Document.....	4
2. Problem Statement and Objectives.....	5
2.1 Problem Statement.....	5
2.2 Problem Analysis.....	5
2.3 Objectives.....	5
2.4 Requirement Coverage.....	5
3. Analysis of Existing Systems.....	6
3.1 Existing Approaches.....	6
3.2 Gap Analysis.....	6
3.3 Proposed Approach.....	6
4. Requirements Analysis.....	7
4.1 User Roles.....	7
4.2 Functional Requirements.....	7
4.3 Non-Functional Requirements.....	7
4.4 Software and Hardware Requirements.....	8
5. System Architecture.....	9
5.1 Architectural Overview.....	9
5.2 Module Responsibilities.....	9
5.3 End-to-End Data Flow.....	9
5.4 Session State.....	10
5.5 Design Decisions.....	10
6. Technology Stack.....	11
6.1 Frontend.....	11
6.2 Application Logic.....	11
6.3 Database and Storage.....	11
6.4 External APIs.....	11
6.5 Development Tools.....	11
7. Data Collection and Knowledge Base.....	12
7.1 Sources and Collection Methods.....	12
7.2 Fields Captured.....	12
7.3 Preparation Steps.....	13
7.4 Handling Missing Data and Service Failure.....	13
7.5 Exploratory Summary.....	13
8. Recommendation, Cost and Language-Processing Module.....	14
8.1 Purpose.....	14
8.2 Request Parsing.....	14
8.3 Recommendation Scoring.....	14
8.4 Cost Estimation.....	14
8.5 Itinerary Generation.....	14
8.6 Retrieval and Answer Generation.....	15
8.7 Parsing Example.....	15
8.8 Evaluation Approach.....	15
9. Database Design.....	16
9.1 Overview.....	16
9.2 Core Tables.....	16
9.3 Relationships.....	16
9.4 Indexing Strategy.....	16
9.5 Migrations.....	16
9.6 Sample Schema Definition (Planned).....	16
9.7 Planned Analytics Queries.....	17
10. Module Interface Design.....	18
10.1 Core Functions.....	18

10.2 LLM Service Interface.....	18
10.3 Error Handling.....	18
10.4 Planned REST Interface.....	18
11. User Interface Design.....	19
11.1 Views.....	19
11.2 Budget Warning Scale.....	19
11.3 Usability and Accessibility.....	19
11.4 Loading and Offline Behaviour.....	19
12. Administration and Content Management (Planned).....	20
12.1 Purpose.....	20
12.2 Planned Features.....	20
12.3 Content Workflow.....	20
13. AI Assistant Workflow.....	21
13.1 Architecture.....	21
13.2 Example.....	21
13.3 Reliability.....	21
14. Security and Privacy.....	22
15. Deployment.....	23
15.1 Local Deployment.....	23
15.2 Deployment Steps.....	23
15.3 Hosting Options.....	23
15.4 Troubleshooting.....	23
16. Testing Strategy.....	24
16.1 Levels of Testing.....	24
16.2 Automated Checks.....	24
16.3 Parser Test Cases.....	24
16.4 Scenario Results.....	24
16.5 System Test Cases.....	26
16.6 Sensitivity Analysis.....	26
17. Scalability and Performance.....	28
17.1 Current Characteristics.....	28
17.2 Planned Scaling Path.....	28
18. Limitations and Future Scope.....	29
18.1 Limitations.....	29
18.2 Future Scope.....	29
19. Conclusion.....	30
References.....	31
Appendix A: Glossary.....	32
Appendix B: Configuration and Project Layout.....	33

1. Introduction

1.1 Background

India is one of the most varied travel destinations in the world. A single trip can involve beaches in Goa, hill stations in Kerala and Karnataka, forts in Rajasthan, mountain adventures in Himachal Pradesh and heritage cities such as Mysuru and Hyderabad. Information needed to plan such a trip, including places to visit, costs, best seasons, local food and safety advice, is available online but is spread across many websites and applications.

A traveller therefore has to combine information from several sources, decide whether the total cost fits the budget, and keep track of spending during the trip. TripEase is designed to bring these steps into one platform that understands a request written in everyday language, recommends destinations, builds a day-by-day plan, estimates the cost and answers questions from a curated knowledge base.

1.2 Motivation

Travel planning is often done by repeating the same preferences in several tools and reconciling the results by hand. General-purpose chatbots can produce attractive plans quickly, but they may state incorrect prices or facts, and they do not check the plan against the traveller's budget. The project is motivated by the need for a transparent planner in which every number can be traced to stored data or an explicit formula, and every factual answer is retrieved from a trusted knowledge base before it is phrased.

1.3 Scope of the Document

This document describes the design and current implementation of the TripEase platform: its requirements, architecture, technology stack, data preparation, recommendation and language-processing modules, data model, module interfaces, user interface, security, deployment and testing approach. It also identifies the components that are designed but not yet implemented. It is intended for evaluators, developers and future maintainers of the system.

1.4 Organisation of the Document

Chapter	Content
1-3	Introduction, problem statement, objectives and analysis of existing systems
4	Functional and non-functional requirements
5-6	System architecture and technology stack
7-8	Data collection and the recommendation, cost and language-processing modules
9-10	Database design and module interface design
11-13	User interface, administration and the AI assistant
14-17	Security, deployment, testing and scalability
18-19	Limitations, future scope and conclusion

Table 1.1: Document organisation

2. Problem Statement and Objectives

2.1 Problem Statement

Design and develop an intelligent tourism platform for India that lets a user describe a trip in natural language and receive personalised destination recommendations, a day-wise itinerary, a cost estimate that respects the stated budget, an expense tracker and a travel assistant grounded in verified tourism information. The platform must keep working, with clear notice, when external services such as the language-model API are unavailable.

2.2 Problem Analysis

- Fragmentation: destination discovery, costs, itineraries, expenses and travel advice are spread over different applications.
- Budget blindness: many planners recommend places without checking whether the total cost fits the traveller's money.
- Unreliable answers: unrestricted chatbots may invent prices, timings or safety information.
- Cold start: a new product has no user history, so recommendations must work from stated preferences alone.
- Dependence on external services: a planner that stops working without an API key is hard to demonstrate and test.

2.3 Objectives

No.	Objective
1	Convert a free-text travel request into structured preferences: duration, budget, travelers, origin and interests.
2	Recommend and rank destinations that match interests and budget, and give a reason for each recommendation.
3	Generate a day-wise itinerary with morning, afternoon and evening activities, a food suggestion and a safety tip, without repeating places.
4	Estimate trip cost by category using transparent formulas and update it when the itinerary changes.
5	Provide an expense tracker that warns when 80% of the budget has been used.
6	Provide an assistant that answers from retrieved knowledge rather than from a language model alone.
7	Remain usable in demo mode without an API key.

Table 2.1: Project objectives

2.4 Requirement Coverage

Problem area	How TripEase addresses it
Free-text request	Rule-based parser extracts days, budget, travelers, origin, interests, state and destination
Personalised ranking	Cosine-similarity content-based recommender with budget and rating terms and a reason per result
Itinerary planning	Non-repeating day-wise plan with override-based editing
Budget control	NumPy cost model, expense form and 80% warning
Grounded answers	TF-IDF retrieval over knowledge documents, optional LLM phrasing
Service outage	Automatic fallback to retrieved facts with a demo-mode notice

Table 2.2: Mapping of requirements to solution components

3. Analysis of Existing Systems

3.1 Existing Approaches

System	Strengths	Limitations
Booking and review portals	Large listings, ratings, booking	User must filter manually; no combined plan or budget check
General-purpose chatbots	Fluent itineraries from a prompt	May invent facts; constraints must be repeated; no cost model
Expense-tracking apps	Accurate spending records	Not linked to destinations or the itinerary
Travel blogs and guides	Rich local detail	Unstructured, inconsistent, not personalised

Table 3.1: Comparison of existing tourism planning approaches

3.2 Gap Analysis

The gap is not a lack of information but a lack of a connected, trustworthy workflow. What is missing is a platform in which (a) a request in plain language becomes structured data, (b) recommendations are explained, (c) the plan is checked against the budget, and (d) factual answers come from a curated source. Existing tools each cover one or two of these steps.

3.3 Proposed Approach

TripEase follows a five-stage pipeline: Understand, Recommend, Plan, Estimate and Assist. The request is parsed into preferences; destinations are ranked with reasons; an itinerary is built; the cost is computed from the itinerary; and the assistant answers questions by retrieving from the knowledge base. Each stage is a separate function, which keeps the system easy to explain, test and extend.

4. Requirements Analysis

4.1 User Roles

Role	Description	Status
Tourist	Plans trips, edits itineraries, tracks expenses, asks the assistant	Implemented
Local business	Lists hotels, restaurants and activities	Planned
Travel administrator	Manages destinations, reviews and analytics	Planned

Table 4.1: User roles

4.2 Functional Requirements

ID	Requirement
FR1	The system shall accept a natural-language trip request and extract duration, budget, travelers, origin, interests and destination.
FR2	The system shall show the extracted values in editable fields so the user can correct them.
FR3	The system shall rank destinations by a combined score and display a reason for each result.
FR4	The system shall restrict ranking to a state when the user names one, such as Karnataka.
FR5	The system shall generate a day-wise itinerary in which no place is repeated.
FR6	The system shall estimate cost by category for the chosen transport mode and stay level.
FR7	The system shall modify a given day from an instruction such as "Change day 3 to include more adventure" and recompute the cost.
FR8	The system shall record expenses and warn when 80% of the budget is used.
FR9	The system shall answer travel questions using information retrieved from the knowledge base.
FR10	The system shall continue to work with retrieved facts when the LLM API is unavailable, and shall say so.
FR11	The system shall allow trips to be saved and listed.
FR12	The system shall display destination analytics through tables and charts.

Table 4.2: Functional requirements

4.3 Non-Functional Requirements

Attribute	Requirement
Reliability	No crash when the API key is missing or the LLM call fails; fall back to retrieved facts
Explainability	Every score and cost traceable to stored data or a stated formula
Performance	Recommendation and itinerary computed locally in well under one second
Security	API key in environment file only; numeric inputs bounded; .env excluded from version control
Usability	Tabbed single-page interface; editable extracted parameters; clear warnings
Maintainability	Modular functions; knowledge base separated from logic
Data integrity	No invented business names; prices marked as approximate

Table 4.3: Non-functional requirements

4.4 Software and Hardware Requirements

Item	Specification
Runtime	Python 3.10 or later
Libraries	streamlit, pandas, numpy, scikit-learn, matplotlib, anthropic, python-dotenv
Browser	Current version of Chrome, Firefox, Edge or Safari
Development machine	Any laptop with 4 GB RAM or more and 1 GB free storage
Network	Internet needed only for pip installation and the optional LLM API

Table 4.4: Software and hardware requirements

5. System Architecture

5.1 Architectural Overview

TripEase is a single-application system with a clear separation of layers. A Streamlit presentation layer handles input and display. A logic layer holds four cooperating modules: the request parser, the recommendation engine, the itinerary builder and the cost estimator. A retrieval module supplies facts to the assistant, and an optional LLM API phrases the final answer. All modules read from a common knowledge base.

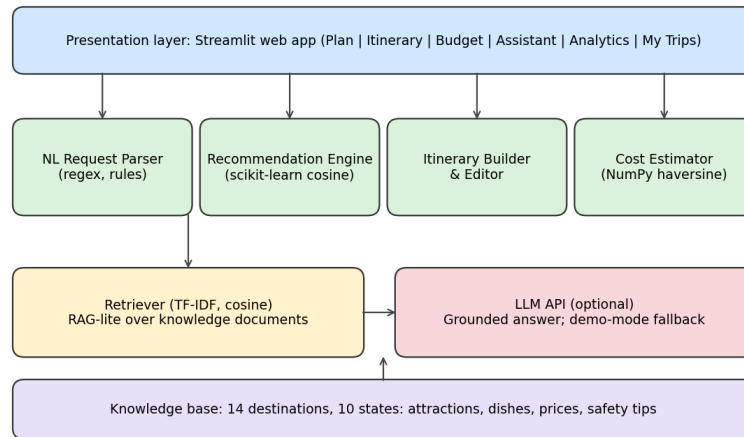


Figure 5.1: Layered architecture of TripEase

5.2 Module Responsibilities

Module	Technology	Responsibility
Presentation	Streamlit	Tabs, forms, metrics, charts, chat and session state
Request parser	Python regular expressions	Extract structured preferences from text
Recommender	Scikit-learn, NumPy	Cosine similarity, budget fit, rating; reasons
Cost estimator	NumPy	Haversine distance and cost by category
Itinerary builder	Python	Non-repeating plan, day overrides
Retriever	Scikit-learn TF-IDF	Find relevant knowledge documents
LLM service	Anthropic API (optional)	Phrase grounded answers
Knowledge base	Python data structures, pandas	Destinations, attractions, dishes, prices, tips

Table 5.1: Components and responsibilities

5.3 End-to-End Data Flow

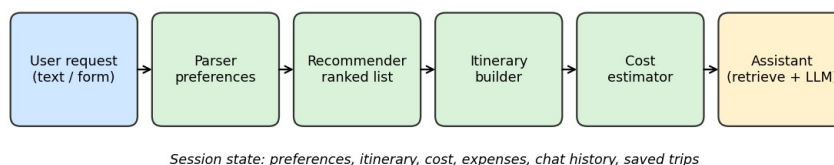


Figure 5.2: End-to-end processing pipeline

- The user enters a sentence or uses the form fields.

- The parser converts the sentence into a preference record that the user can edit.
- The recommender scores every eligible destination and returns a ranked table with reasons.
- The user selects a destination, transport mode and stay level; the itinerary builder creates the plan.
- The estimator computes cost using the places actually chosen; edits rebuild the plan and recompute the cost.
- Questions are answered by retrieving documents and, if a key is present, phrasing the answer with the LLM.

5.4 Session State

Key	Content
p	Extracted preferences: days, budget, travelers, origin, interests
dest, mode, stay	Selected destination, transport mode and stay level
it, override	Current itinerary and stored day overrides
est	Latest cost estimate by category
expenses	Expense records entered by the user
chat	Conversation history with the assistant
trips	Saved trips

Table 5.2: Session state keys

5.5 Design Decisions

- A rule-based parser was chosen for the main fields because it is fast, deterministic and easy to test.
- A content-based recommender was chosen because a new product has no interaction history for collaborative filtering.
- The cost model is rule-based and transparent so each figure can be traced; it can later be replaced by a trained regression model.
- Retrieval happens before generation so that answers are limited to the knowledge base.
- Everything runs locally, and the external LLM is optional, so the project can be demonstrated without any account.

6. Technology Stack

6.1 Frontend

Layer	Technology	Purpose
Web interface	Streamlit	Six tabs, forms, metrics, chat, expanders, tables
Charts	Matplotlib and Streamlit charts	Analytics and expense charts
State	Streamlit session state	Preferences, itinerary, expenses, chat, saved trips

Table 6.1: Frontend technologies

6.2 Application Logic

Library	Use in TripEase
Python 3	Parsing, orchestration, itinerary logic
NumPy	Haversine distance, vector operations, cost arithmetic
pandas	Ranked tables, expense tables, analytics data
Scikit-learn	Cosine similarity, TF-IDF vectorizer
python-dotenv	Loads environment variables from .env

Table 6.2: Application libraries

6.3 Database and Storage

Component	Current	Planned
Tourism data	Python knowledge base (14 destinations)	PostgreSQL tables with keys and indexes
User trips and expenses	Session state	PostgreSQL; MySQL for practice queries
Analytics	Charts in the Analytics tab	SQL views feeding Power BI
Retrieval index	TF-IDF built in memory	Embeddings in a vector database

Table 6.3: Data storage

6.4 External APIs

Service	Use	Status
Anthropic LLM API	Phrase answers from retrieved context	Optional; key in .env
Hugging Face models	Review sentiment and embeddings	Planned

Table 6.4: External services

6.5 Development Tools

Development uses Visual Studio Code and Jupyter for exploration. Version control is with Git and GitHub. Dependencies are installed with pip from requirements.txt.

7. Data Collection and Knowledge Base

7.1 Sources and Collection Methods

The knowledge base is curated by the team from widely known tourism information about each destination. It contains 14 destinations across Goa, Karnataka, Kerala, Tamil Nadu, Rajasthan, Himachal Pradesh, Uttarakhand and Telangana, with 64 attractions and activities. Prices and ratings are approximate planning values for a budget traveller and are labelled as such. Named hotel and restaurant businesses are deliberately not included, because inventing them would mislead users.

Destination	State	Interest tags	Rating	Room ₹/night	Food ₹/day	Best time
Goa	Goa	beach, food, adventure, culture	4.5	1,500	600	Nov-Feb
Gokarna	Karnataka	beach, spiritual, nature, adventure	4.3	1,000	400	Oct-Mar
Udupi	Karnataka	food, beach, spiritual, culture	4.2	1,000	400	Oct-Feb
Coorg	Karnataka	nature, food, adventure, culture	4.4	1,500	500	Oct-Mar
Mysuru	Karnataka	history, culture, food	4.5	1,200	450	Oct-Feb
Hampi	Karnataka	history, culture, adventure	4.6	1,100	400	Oct-Feb
Munnar	Kerala	nature, adventure, food	4.5	1,500	500	Sep-Mar
Varkala	Kerala	beach, spiritual, food	4.3	1,300	500	Nov-Mar
Alleppey	Kerala	nature, food, culture	4.4	1,500	500	Nov-Feb
Pondicherry	Tamil Nadu	beach, food, culture, spiritual	4.3	1,500	600	Oct-Mar
Jaipur	Rajasthan	history, culture, food	4.5	1,500	600	Oct-Mar
Manali	Himachal Pradesh	adventure, nature	4.4	1,500	550	Mar-Jun, Oct-Nov
Rishikesh	Uttarakhand	adventure, spiritual, nature	4.4	1,000	400	Sep-Nov, Feb-Apr
Hyderabad	Telangana	food, history, culture	4.4	1,500	700	Oct-Feb

Table 7.1: Destinations in the knowledge base

7.2 Fields Captured

Field	Description
State and coordinates	State name, latitude and longitude for distance calculation
Interest tags	beach, food, adventure, culture, history, nature, spiritual
Room rate and food cost	Budget room per night; food per person per day (₹)
Rating and best time	Curated rating out of 5; recommended months
Attractions	Name, cost per person and interest type
Dishes and tip	Signature local dishes and a safety or local tip

Table 7.2: Data fields per destination

7.3 Preparation Steps

- Each destination is stored as a structured record and converted to a pandas table for display.
- Interest tags are converted to binary vectors over seven categories for similarity calculation.
- The data is turned into 56 short retrieval documents (food, attractions, safety and tips, best time).
- Costs are stored as numbers so that totals can be computed and rounded consistently.

7.4 Handling Missing Data and Service Failure

If an interest is not tagged for any destination, the recommender falls back to all interests so it still returns results. If the LLM key is absent or the call fails, the assistant returns the retrieved facts and the interface states that it is running in demo mode. No fabricated values are inserted to fill gaps.

7.5 Exploratory Summary

Across the 14 destinations, the mean rating is 4.41, the mean budget room rate is ₹1,328 per night and the mean food cost is ₹507 per person per day. Figure 7.1 shows the interest tags carried by each destination.

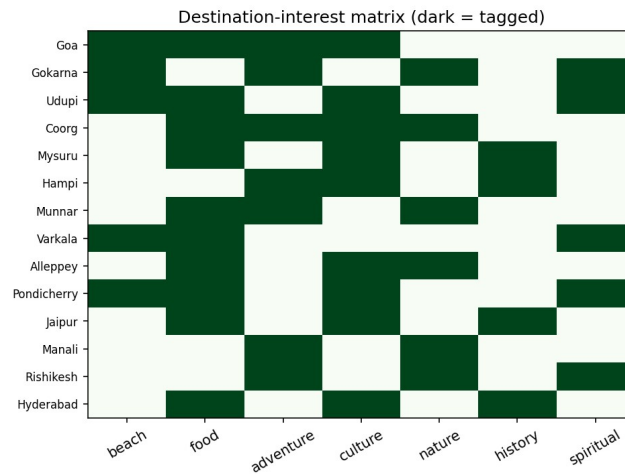


Figure 7.1: Destination and interest matrix

The interest counts show that food is the most widely covered interest (10 destinations) and history the least covered (4). An interest tagged in few destinations gives the user fewer options, so coverage will be extended as the knowledge base grows.

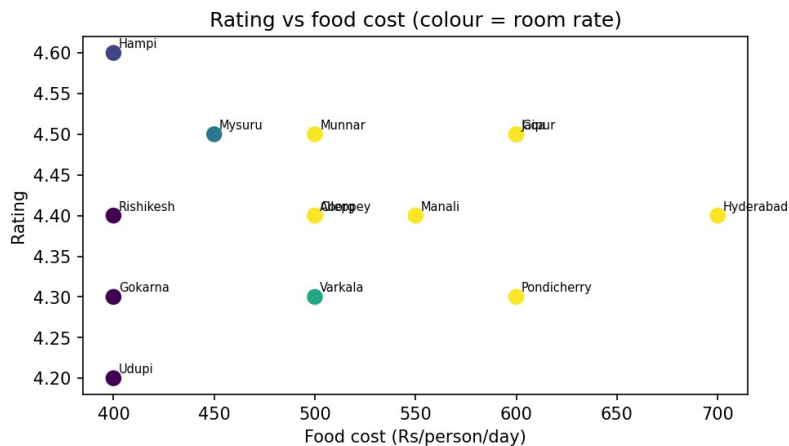


Figure 7.2: Rating versus food cost, coloured by room rate

The scatter plot shows that higher room rates do not necessarily accompany higher ratings, so a budget-aware recommender can find well-rated places at moderate cost.

8. Recommendation, Cost and Language-Processing Module

8.1 Purpose

This module converts a request into ranked, explained and costed plans. It combines rule-based language processing, content-based machine learning and numerical modelling, and it produces the outputs shown on the Plan and Itinerary tabs.

8.2 Request Parsing

The parser lowercases the request and uses regular expressions and keyword dictionaries. It bounds days to 1-14 and travelers to 1-12, understands "15k" as 15,000, normalises "Bangalore" to Bengaluru, and does not mistake "for 2 days" for a group size.

Field	Rule	Default
Days	number followed by "day"	3
Budget	under / below / within / budget of / ₹ / Rs / INR plus number, optional "k"	₹15,000
Travelers	number + people/persons/adults/friends, or "for" + number word	2
Origin	"from" + known city	Bengaluru
Interests	keyword dictionary, seven categories	all
State / destination	name found in the text	none

Table 8.1: Parsing rules

8.3 Recommendation Scoring

Each destination is a binary vector over seven interest categories and the user's interests form a query vector. Cosine similarity from Scikit-learn gives the interest match. The final score is:

$$\text{Score} = 0.5 \times \text{cosine similarity} + 0.3 \times \text{budget fit} + 0.2 \times (\text{rating} - 3) / 2$$

Budget fit is 1 when the estimate is within budget and falls linearly to 0 at twice the budget. For Goa with the interests beach, food and adventure, cosine similarity is $3 / (\sqrt{3} \times 2) = 0.866$; with an estimate of ₹20,060 against ₹15,000 the budget fit is 0.663; the rating term is 0.75; the score is 0.782, matching the program output.

8.4 Cost Estimation

- **Hotel:** room rate $\times$ stay factor (budget 1.0, mid 1.8, luxury 3.5) $\times$ (days - 1) $\times$ rooms, one room per two travelers.
- **Food:** food cost per person per day $\times$ days $\times$ travelers.
- **Transport:** round-trip haversine distance $\times$ 1.3 $\times$ rate per km (bus ₹2.2, train ₹1.3, flight ₹6.0, car ₹11.0 per vehicle); flights add ₹1,500 per person.
- **Activities:** sum of costs of the places chosen in the itinerary, times travelers.
- **Other:** 10% contingency on the subtotal.

8.5 Itinerary Generation

Places are sorted by match with the user's interests. Each day gets up to three slots, morning, afternoon and evening. Each place is used at most once; unused slots become free time. An edit such as "Change day 3 to include more adventure" stores an override for that day, gives it the requested kind of place first, rebuilds the plan and keeps earlier overrides.

8.6 Retrieval and Answer Generation

The knowledge base yields 56 short documents. For a question, TF-IDF vectors are computed and the three most similar documents are selected. With an API key, the documents and the trip summary are sent to the LLM with an instruction to answer only from the context. Without a key, the documents themselves are returned.

8.7 Parsing Example

Extracted field	Value from Scenario 1
Days	4
Budget	₹15,000
Travelers	2
Origin	Bengaluru
Interests	beach, food, adventure
Destination	Goa

Table 8.2: Preferences extracted from the Goa request

8.8 Evaluation Approach

Correctness is checked by scripted tests (Chapter 16): itinerary validity for all destinations and durations, cost consistency, monotonic behaviour of cost, retrieval relevance and parser accuracy. Recommendation quality is inspected through worked examples and sensitivity analyses in Chapter 16.

9. Database Design

9.1 Overview

The current version holds its data in Python structures and session state. The following normalised PostgreSQL schema is the planned design for persistent storage and analytics. It is a design and has not been implemented yet.

9.2 Core Tables

Table	Key columns	Notes
users	user_id (PK), name, email (unique), password_hash, role	Roles: tourist, business, admin
destinations	destination_id (PK), name, state, latitude, longitude, rating, best_time	Parent of attractions and hotels
attractions	attraction_id (PK), destination_id (FK), name, category, cost	Interest category per place
hotels	hotel_id (PK), destination_id (FK), name, price_per_night, rating	Real listings to be added
restaurants	restaurant_id (PK), destination_id (FK), name, cuisine, avg_price	Real listings to be added
trips	trip_id (PK), user_id (FK), destination_id (FK), start_date, days, budget	Saved trips
itineraries	itinerary_id (PK), trip_id (FK), day, slot, attraction_id (FK)	Day-wise plan
expenses	expense_id (PK), trip_id (FK), category, amount, spent_on	Budget tracking
reviews	review_id (PK), destination_id (FK), user_id (FK), rating, text, sentiment	Sentiment from NLP model

Table 9.1: Planned tables

9.3 Relationships

- One destination has many attractions, hotels and restaurants.
- One user has many trips; one trip has many itinerary rows and many expenses.
- One user and one destination are linked through many reviews.

9.4 Indexing Strategy

Foreign-key columns would be indexed, along with destinations.state, trips.user_id, expenses.trip_id and reviews.destination_id. Analytics views such as destination performance and monthly trends would be built over these tables for Power BI.

9.5 Migrations

Schema creation and seed data would be kept in versioned SQL scripts (schema, seed and analytics queries) so the database can be rebuilt on any machine. All queries would use parameterised statements to prevent SQL injection.

9.6 Sample Schema Definition (Planned)

The excerpt below shows how the trips and expenses tables would be declared, with primary keys, foreign keys, constraints and indexes.

```
CREATE TABLE trips (
  trip_id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(user_id),
```

```

    destination_id INT NOT NULL REFERENCES destinations(destination_id),
    start_date     DATE,
    days           INT CHECK (days BETWEEN 1 AND 14),
    budget         NUMERIC(10,2) CHECK (budget > 0)
);

CREATE TABLE expenses (
    expense_id SERIAL PRIMARY KEY,
    trip_id     INT NOT NULL REFERENCES trips(trip_id) ON DELETE CASCADE,
    category    VARCHAR(20) NOT NULL,
    amount      NUMERIC(10,2) CHECK (amount > 0),
    spent_on    DATE DEFAULT CURRENT_DATE
);

CREATE INDEX idx_expenses_trip ON expenses(trip_id);

```

9.7 Planned Analytics Queries

Question	Query idea
Top destinations	Count trips grouped by destination, ordered by count
Average spend per trip	Average of summed expenses per trip, grouped by destination
Monthly demand	Trips grouped by month of start date
Popular activities	Itinerary rows grouped by attraction category

Table 9.2: Planned analytics queries

Each of these would be exposed as a SQL view so that Power BI can read prepared data without running heavy queries on the operational tables.

10. Module Interface Design

10.1 Core Functions

Function	Input	Output
parse(text)	Trip request text	Dictionary: days, budget, travelers, origin, interests, states, dest
recommend(prefs, top)	Preference dictionary	DataFrame with score, estimated cost, reason
estimate(dest, days, n, origin, mode, stay, acts)	Trip parameters	Cost by category, distance and mode
itinerary(dest, days, interests, n, override)	Destination and interests	List of days with slots, food and tip
modify(...)	Edit request and current overrides	New itinerary, overrides, message
trip_cost(it)	Itinerary	Total activity cost
retrieve(query, k)	Question text	Top documents
ask(question, context)	Question and trip summary	Answer, documents, demo-mode flag

Table 10.1: Core module interfaces

10.2 LLM Service Interface

The `llm()` function sends the system instruction and the prompt to the Anthropic API using the key and model name from the environment. It returns the text or `None` if the key is missing or the call fails, which triggers the fallback path.

10.3 Error Handling

- Missing API key or failed call: the assistant returns retrieved facts and shows a demo-mode note.
- Unrecognised day or interest in an edit: a helpful message describes the expected form.
- No matching documents: the assistant says it has no information instead of guessing.

10.4 Planned REST Interface

If the platform is split into a web front end and a service, the functions above map naturally to endpoints such as `POST /parse`, `GET /recommendations`, `POST /itineraries`, `POST /estimates` and `POST /assistant`. This is a possible extension and is not part of the current version.

11. User Interface Design

11.1 Views

Tab	Content
Plan	Text request, editable parameters, ranked recommendations with reasons, destination, transport and stay selection
Itinerary	Day-wise plan, cost metrics by category, edit box, Save trip
Budget	Expense form, total / spent / remaining metrics, warning at 80%, category chart
Yatra Assistant	Chat with retrieval-grounded answers
Analytics	Destination table, rating chart, average room price by state
My Trips	Saved trips with destination, duration, budget and estimate

Table 11.1: Application views

11.2 Budget Warning Scale

Budget used	Message style
Below 80%	Information message showing the percentage used
80% and above	Warning message: "You have used N% of your planned budget."

Table 11.2: Budget alerts

11.3 Usability and Accessibility

- Extracted values appear in editable fields so mistakes can be corrected.
- Costs are shown as clear metric cards by category.
- Text is readable on light backgrounds and charts have titles and axis labels.
- The layout is single-column friendly, which suits narrow screens as far as Streamlit allows.

11.4 Loading and Offline Behaviour

All planning computations are local and immediate. Only the optional LLM call needs the network; if it fails, retrieved facts are shown and the interface says so.

12. Administration and Content Management (Planned)

12.1 Purpose

An administrator role is part of the design to keep tourism data accurate. It is not implemented in the current version and is described here as the planned extension.

12.2 Planned Features

- Add or update destinations, attractions, hotels and restaurants.
- View and moderate reviews and their sentiment.
- View users and active trips.
- See dashboards of popular destinations, demand by month and spending.

12.3 Content Workflow

A business or administrator submits a new listing; validation checks required fields and numeric ranges; the record is stored with a timestamp and the identity of the editor; the recommender and retrieval index are refreshed from the updated tables.

13. AI Assistant Workflow

13.1 Architecture

The Yatra Assistant receives the user's question and a short summary of the current trip. The retriever selects supporting documents. If the LLM is available, it answers using only that context; otherwise the documents are returned. Spending questions are answered directly from the stored expenses.

13.2 Example

Question: "What local food should I try in Goa?"

Retrieved answer (demo mode): Goa best time to visit: Nov-Feb. State: Goa.

13.3 Reliability

- The answer is always limited to the knowledge base.
- If nothing relevant is found, the assistant says so.
- The interface shows whether it is using the LLM or demo mode.

14. Security and Privacy

Area	Measure	Status
Secrets	API key loaded from .env; .env excluded from Git; .env.example provided	Implemented
Input validation	Bounded numeric inputs; parser clamps days and travelers	Implemented
Error handling	Exceptions around LLM calls; fallback path	Implemented
Authentication	Password hashing, sessions	Planned
Access control	Role-based access for tourist, business and admin	Planned
SQL safety	Parameterised queries	Planned with database

Table 14.1: Security measures

The current version stores no personal data beyond the browser session. When accounts are added, only password hashes will be stored, and trip data will be visible only to its owner.

15. Deployment

15.1 Local Deployment

TripEase runs on any machine with Python 3.10 or later. It needs no database or external service for the core features, which makes it easy to demonstrate.

15.2 Deployment Steps

Step	Action
1	Clone the repository and create a virtual environment
2	Install dependencies with <code>pip install -r requirements.txt</code>
3	Optionally copy <code>.env.example</code> to <code>.env</code> and add <code>ANTHROPIC_API_KEY</code>
4	Run <code>streamlit run app.py</code>
5	Open <code>http://localhost:8501</code> in a browser

Table 15.1: Deployment steps

15.3 Hosting Options

The application can be hosted on a Streamlit-compatible service or in a container. Container images and a PostgreSQL service are planned with the database phase; the API key would then be supplied through the hosting platform's secret store.

15.4 Troubleshooting

Problem	Likely cause	Action
ModuleNotFoundError	Dependencies not installed in the active environment	Activate the virtual environment and run <code>pip install -r requirements.txt</code>
Assistant shows only facts	No <code>ANTHROPIC_API_KEY</code> set, or the API call failed	Add the key to <code>.env</code> and restart; demo mode is expected without it
Port 8501 already in use	Another Streamlit app is running	Stop it or run <code>streamlit run app.py --server.port 8502</code>
Unexpected parsed values	Unusual wording in the request	Correct the values in the editable fields on the Plan tab

Table 15.2: Common problems and actions

16. Testing Strategy

16.1 Levels of Testing

Level	Focus	Status
Function checks	Parser, estimator, itinerary, retrieval	Done (scripted)
Interface checks	Streamlit headless run of Plan, Itinerary and edit actions	Done
Unit tests	Formal pytest suite	Planned
User testing	Feedback from students and travellers	Planned

Table 16.1: Testing levels

16.2 Automated Checks

A set of 148 scripted checks was run against the implemented functions, and all passed.

Check	Cases	Passed
Itinerary has the requested days and no repeated place (14 destinations × 1-7 days)	98	98
Estimated total equals the sum of its components	14	14
Cost rises with more days and more travelers	14	14
Food query retrieves a document about the queried destination	14	14
Parser extracts expected fields	8	8

Table 16.2: Automated checks

16.3 Parser Test Cases

Request	Parsed	Result
Plan a 3-day Mysuru trip under ₹8,000 for 2 people. I like history,...	3d, ₹8,000, 2 trav., Bengaluru, Mysuru	Pass
Weekend in Hampi 2 days Rs 5,000 for one	2d, ₹5,000, 1 trav., Bengaluru, Hampi	Pass
7 day trip from Delhi to Manali under 25k with 4 friends, adventure...	7d, ₹25,000, 4 trav., Delhi, Manali	Pass
5-day Kerala trip under ₹20,000 for three, backwaters and food	5d, ₹20,000, 3 trav., Bengaluru, None	Pass
from Mumbai to Goa 3 days budget of 12000 for 2 adults beach	3d, ₹12,000, 2 trav., Mumbai, Goa	Pass
trip from Bangalore to Coorg for 2 days under 6k, coffee and nature	2d, ₹6,000, 2 trav., Bengaluru, Coorg	Pass
20 day trip to Jaipur	14d, ₹15,000, 2 trav., Bengaluru, Jaipur	Pass
Jaipur trip 3 days	3d, ₹15,000, 2 trav., Bengaluru, Jaipur	Pass

Table 16.3: Parser test cases

16.4 Scenario Results

Scenario 1: "Plan a 4-day trip from Bengaluru to Goa under ₹15,000 for two people. I want beaches, local food and adventure."

Rank	Destination	State	Score	Est. cost (₹)
1	Goa	Goa	0.782	20,060
2	Munnar	Kerala	0.780	15,190
3	Coorg	Karnataka	0.729	14,500
4	Gokarna	Karnataka	0.719	11,840
5	Udupi	Karnataka	0.709	11,780

Table 16.4: Recommendations for Scenario 1

Component	Amount
Hotel	₹4,500
Food	₹4,800
Transport (596 km by bus)	₹5,250
Activities	₹4,300
Other (10%)	₹1,880
Total	₹20,730

Table 16.5: Goa cost estimate, 4 days, 2 travelers

Goa matches all three interests but exceeds the budget; the interface says so and lists alternatives such as Coorg, Gokarna and Udupi that fit.

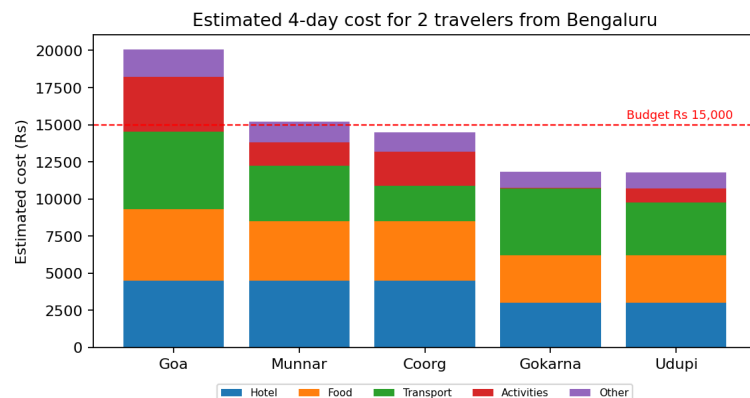


Figure 16.1: Estimated cost by category for the top five destinations

Day	Morning	Afternoon	Evening	Food
1	Baga & Calangute beaches (₹0)	Palolem beach (₹0)	Free time (₹0)	Goan fish thali
2	Old Goa churches (UNESCO) (₹0)	Fort Aguada (₹100)	Free time (₹0)	Pork vindaloo
3	Water sports at Baga (₹3,000)	Dudhsagar Falls jeep safari (₹1,200)	Free time (₹0)	Bebinca dessert
4	Fontainhas Latin Quarter walk (₹0)	Free time (₹0)	Free time (₹0)	Prawn balchao

Table 16.6: Goa itinerary after "Change day 3 to include more adventure"

Scenario 2: "4 days in Karnataka under 15k with nature and food" restricts ranking to Karnataka.

Rank	Destination	State	Score	Est. cost (₹)
1	Coorg	Karnataka	0.794	14,500
2	Mysuru	Karnataka	0.654	10,180
3	Gokarna	Karnataka	0.607	11,840
4	Udupi	Karnataka	0.597	11,780
5	Hampi	Karnataka	0.460	11,700

Table 16.7: Recommendations for Scenario 2

16.5 System Test Cases

ID	Input / action	Expected result	Result
TC1	Goa request with "under ₹15,000 for two people"	Days 4, budget 15000, 2 travelers, Goa	Pass
TC2	"4 days in Karnataka under 15k with nature and food"	Budget 15000; only Karnataka destinations ranked	Pass
TC3	"20 day trip to Jaipur"	Days limited to 14	Pass
TC4	Itinerary for every destination, 1-7 days	Requested days; no repeated place	Pass
TC5	"Change day 3 to include more adventure"	Day 3 shows adventure places; other days unchanged	Pass
TC6	Cost with more days or more travelers	Total increases	Pass
TC7	Food question for each destination	Document about that destination retrieved	Pass
TC8	Streamlit headless run of Plan, Generate and edit	No exceptions	Pass

Table 16.8: System test cases

16.6 Sensitivity Analysis

Mode	Transport (₹)	Total (₹)
bus	5,250	20,060
train	3,100	17,690
flight	17,310	33,320
car	13,120	28,710

Table 16.9: Goa trip by transport mode (from Bengaluru)

Origin	Distance (km)	Mode	Total (₹)
Bengaluru	596	bus	20,060
Mumbai	570	bus	19,810
Delhi	1,968	bus	33,340
Chennai	924	bus	23,240
Hyderabad	682	bus	20,890
Kochi	835	bus	22,370
Mangaluru	362	bus	17,790

Origin	Distance (km)	Mode	Total (₹)
Pune	466	bus	18,800

Table 16.10: Goa trip by origin city

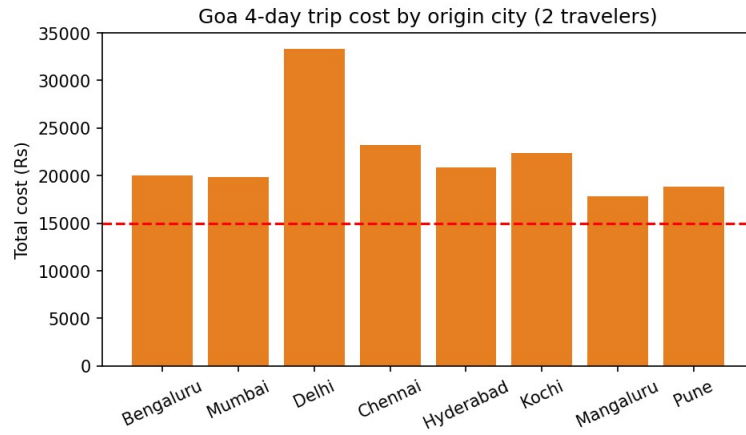


Figure 16.2: Goa trip cost by origin city

Stay level	Hotel (₹)	Total (₹)
budget	4,500	20,060
mid	8,100	24,020
luxury	15,750	32,430

Table 16.11: Goa trip by stay level

Budget	Rank 1	Rank 2	Rank 3
₹8,000	Goa (20,060)	Gokarna (11,840)	Udupi (11,780)
₹15,000	Goa (20,060)	Munnar (15,190)	Coorg (14,500)
₹25,000	Goa (20,060)	Munnar (15,190)	Varkala (19,690)
₹40,000	Goa (20,060)	Munnar (15,190)	Varkala (19,690)

Table 16.12: Top three destinations by budget (estimate in brackets)

At an ₹8,000 budget no strongly matching destination fits, so the ranking is decided mainly by interest match and Goa still appears first while marked as above budget. A stricter budget filter is planned.

17. Scalability and Performance

17.1 Current Characteristics

- All scoring is vectorised over 14 destinations and completes almost instantly.
- The retrieval index is built in memory from 56 short documents.
- The application has no external dependency for core features, so it does not slow down when a network is slow.

17.2 Planned Scaling Path

Need	Approach
Thousands of destinations and listings	Move data to PostgreSQL with indexes; load only filtered rows
Semantic retrieval at scale	Embeddings stored in a vector database
Many concurrent users	Separate service layer, caching of rankings, replicated instances
Learned cost model	Regression trained on real expense data

Table 17.1: Scaling path

18. Limitations and Future Scope

18.1 Limitations

- Only 14 destinations with a small number of places each; some plans include free time.
- Prices and ratings are approximate, not live quotes.
- The cost model is rule-based; no trained regression model yet.
- Recommendations use binary interest tags and do not yet use user history or review sentiment.
- When nothing fits the budget, interest match dominates the ranking.
- TF-IDF retrieval matches words, not meaning.
- Trips are held only in the browser session; there is no login.

18.2 Future Scope

Enhancement	Technology	Benefit
Persistent storage and analytics views	PostgreSQL, MySQL, SQL views	Saved trips and reporting
Tourism analytics dashboard	Power BI	Insights for administrators
Tourist segmentation	Scikit-learn K-Means	Personalisation without permanent labels
Review sentiment	Hugging Face	Quality signal in ranking
Semantic retrieval	Embeddings and a vector database	More accurate grounded answers
Trained cost model	Scikit-learn regression	Learned estimates
Hard budget filter	Python	Over-budget destinations ranked lower
Authentication and roles	Password hashing, RBAC	Multi-user operation
Notebooks and unit tests	Jupyter, Seaborn, pytest	Reproducibility

Table 18.1: Planned enhancements

19. Conclusion

TripEase shows that a single connected workflow can replace several disconnected planning steps. A plain-English request becomes structured preferences, ranked destinations with reasons, a non-repeating day-wise itinerary, a transparent cost estimate that updates when the plan changes, an expense tracker and a knowledge-grounded assistant that still works without an external API.

The current implementation is deliberately compact so that every technique can be explained end to end: haversine distance in NumPy, cosine similarity in Scikit-learn, TF-IDF retrieval, prompt grounding for an LLM and a Streamlit interface. The database, analytics, segmentation and sentiment modules described as planned will extend the same architecture toward a full tourism intelligence platform.

References

- Python Software Foundation. Python 3 documentation. python.org
- NumPy documentation. numpy.org
- pandas documentation. pandas.pydata.org
- Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2011.
- Matplotlib documentation. matplotlib.org
- Streamlit documentation. docs.streamlit.io
- Anthropic API documentation. docs.claude.com
- Sinnott, R. W. Virtues of the Haversine. Sky and Telescope, 68(2), 1984.
- Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- Manning, Raghavan and Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.

Appendix A: Glossary

Term	Meaning
Cosine similarity	Similarity of two vectors from the angle between them
Content-based recommendation	Recommending items whose features match the user's stated preferences
Haversine formula	Great-circle distance between two latitude/longitude points
TF-IDF	Word weighting by frequency in a document and rarity across documents
RAG	Retrieving relevant text first and giving it to a language model as context
Demo mode	Operation without an LLM key, returning retrieved facts

Table A.1: Glossary

Appendix B: Configuration and Project Layout

Path	Description
app.py	Streamlit application entry point
tripease/data.py	Knowledge base
tripease/core.py	parse, recommend, estimate, itinerary, modify, retrieve, ask
requirements.txt	Python dependencies
.env.example	Template for ANTHROPIC_API_KEY and LLM_MODEL
README.md	Run instructions

Table B.1: Project layout

Environment variables: ANTHROPIC_API_KEY enables LLM-phrased answers and LLM_MODEL sets the model name. Both are optional; without them the application runs in demo mode.